DATA 298A MSDA Project I

Workbook 2

Multiplayer Agents: A Shared Live Session Layer for Human-Agent Collaboration

Team - 4
Ayush Gawai, Naman Vipul Chheda, Manav Patel, Shriram Dundigalla, Pramod Satya Dindukurthi
Section 21, Topic 4
Advisor: Prof. Simon Shim
November 11, 2026

Team member	Sections written
Ayush Gawai	4.1, 4.2, 4.3
Naman Vipul Chheda	3.1, 3.2, 3.3
Manav Patel	3.5, 3.6
Shriram Dundigalla	3.4, 4.5
Pramod Satya Dindukurthi	4.4, 4.6

Author allocation for the twelve subsections of this workbook. This column records who wrote each section; the owner column in the self-assessment table records who is accountable for the underlying work, and the two differ where one member wrote up another member's results.

GitHub: https://github.com/ayushgawai/multiplayer-agent-sessions
Linear: https://linear.app/data-298a-team-4/team/DAT/cycles

Workbook 2 is report sections 3 and 4. Sections 1 and 2 are not rewritten here; revisions made to them since Workbook 1 are recorded in the carry-forward table at the end. Table and figure numbering continues from Workbook 1, which ended at Table 17 and Figure 2, and the reference list continues from [17].

3. Data Engineering

3.1 Data Process

This project runs three separate data paths, and keeping them separate is the point rather than an accident of organisation. The first supplies the reproduced published baseline, the second supplies supervision for Model 1, and the third supplies the dataset on which the Model 1 candidate winner is actually selected. Mixing them would make the baseline reproduction and the Model 1 result incomparable, so each path has its own provenance record and its own freeze. Figure 3 shows the three paths from upstream source through freeze to the consuming model or metric.

Baseline path (MAST)

The MAST release is downloaded at a pinned Hugging Face revision by `data/scripts/download_mast.py`, together with the taxonomy reference from the project's GitHub repository at a pinned commit (DAT-28, DAT-29). A frozen evaluation manifest, `data/mast/eval_manifest.json`, then names exactly which file, which label field and which reference

source each run may read (DAT-30). The judge prompt and response parser consume that manifest (DAT-31, DAT-32), the batch runner executes the judge and writes one record per trace (DAT-34, DAT-35), and the metrics layer reads those records (DAT-37). The record shape produced at the end of this path is frozen in ADR-003.

Model 1 supervision path (QMSum)

QMSum is downloaded by `data/scripts/download_qmsum.py` and converted into task examples by `data/scripts/make_splits.py`, which writes `data/splits/m1_v1/` with a manifest carrying a SHA-256 per split. These splits supervise Model 1 only. QMSum is not a baseline, and this project does not attempt to reproduce its published summarization scores.

Candidate-selection path (product sessions)

The frozen experiment contract names `session_test` as the split on which the Model 1 candidates are compared, drawn from annotated product-session data at `data/splits/m1_session_v1/`. That dataset does not exist yet. Until it does, no reportable candidate-selection result can be produced, and the contract fails closed rather than substituting QMSum. This is the single largest open item in section 3 and is recorded in the gap register at the end of this workbook.

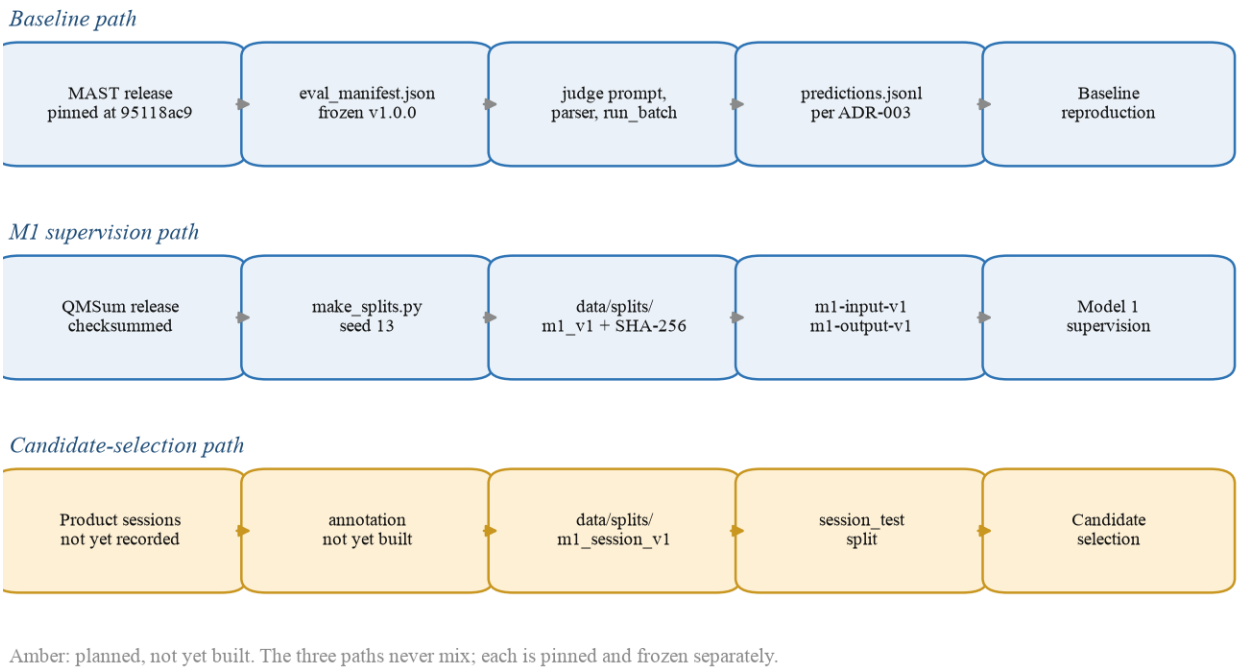


Figure 3. Data lineage for the three paths, from pinned upstream source through the freeze point to the consuming model or metric. Generated by `docs/reports/generate_t4a5_figures.py`.

3.2 Data Collection

Two upstream corpora have been collected. Both are pinned by revision and verified by checksum, so a later run can prove it read the same bytes rather than merely asserting it.

Property	MAST (baseline)	QMSum (Model 1 supervision)
Source	Hugging Face `mcemri/MAD` [3]	Yale-LILY QMSum release [2]

Property	MAST (baseline)	QMSum (Model 1 supervision)
Pinned revision	`95118ac951421753cf1deb87ddea3b01e693c41b`	Upstream release files, checksummed on download
Licence	CC-BY-4.0	MIT
Quantity retrieved	1,642 annotated traces	232 meetings, 1,810 query-summary pairs
Agreement with the paper	1,642 retrieved against 1,642 stated: exact match	Not a baseline; no published number is being matched
Primary file	`MAD_full_dataset.json`, 199,574,367 bytes	`train/val/test.jsonl`, 22.5 MB total
Secondary file	`MAD_human_labelled_dataset.json`, 19 records	—
Taxonomy reference	GitHub commit `a70542e541b2104ef8fcd785778179e173fb8d70`	—
Collected	25 September 2026	22 October 2026

Table 18. Sources, pinning and quantity of the two collected corpora. Per-file SHA-256 digests are recorded in `data/mast/STATS.md` and `data/qmsum/STATS.md`.

The 19-record human-labelled MAST file is an inter-annotator agreement study whose taxonomy was revised between annotation rounds. It is not comparable to the 1,642-record file and is never pooled with it, which is the reason requirement DR-8 records a reference source on every prediction.

A raw QMSum record is a meeting transcript with a list of general and specific queries, each paired with a human-written summary. A raw MAST record is a multi-agent execution trace with six top-level fields: `mas_name`, `llm_name`, `benchmark_name`, `trace_id`, `trace` and `mast_annotation`, where the annotation is a map from the fourteen failure codes to a binary label.

[OWNER: Naman] Paste one raw MAST trace record and one raw QMSum meeting record here, truncated, so the reader can see the raw shape before pre-processing.

3.3 Data Pre-processing

Pre-processing on the baseline path is mostly a matter of establishing identity and label semantics correctly, because the upstream release contains several traps that would silently corrupt a score rather than raise an error.

The record key is a pair, not a field

`trace_id` is not unique: there are only 206 distinct values across the 1,642 records, because the identifier resets per source run. The unique key is the pair (`trace.key`, `trace_id`), which the runner materialises as a single identifier of the form `<trace_key>::<trace_index>`. Keying on the raw field alone would have collapsed roughly eight records into one and inflated agreement.

Labels are keyed by code, not display name

The canonical label order is the fourteen MAST codes 1.1 through 3.3. Display names are not used as keys, because they differ between the release and the taxonomy reference.

A numbering conflict is resolved explicitly

The answer template in the judge prompt names codes 3.2 and 3.3 in the opposite order to the taxonomy definitions file. The parser keys labels by the template's numbering, and ADR-003 records this, so the scoring meaning of those two codes is an explicit decision rather than an accident that nobody noticed.

Reference sources are never pooled

The manifest distinguishes `released_llm_annotation` from `human_consensus`, and results measured against each are reported separately. Agreement against the released LLM annotation is a judge-versus-judge comparison and cannot be reported as reproducing the paper's human-evaluation numbers.

On the Model 1 path, pre-processing converts each QMSum query-summary pair into a task example with four fields, `id`, `query`, `context_spans` and `reference_summary`, where the context spans are the transcript turns the upstream annotation marked as relevant to that query.

Rule	Applied to	Rows removed	Rows remaining	Note
Composite key construction	MAST, 1,642 records	0	1,642	Not a removal. Prevents the 206-value `trace_id` collision described above
Null-label audit	MAST, 1,642 records	0	1,642	56 null label cells found, including 4 fully-null records. Recorded in `data/mast/AUDIT_REPORT.md`, not dropped
Manifest scoring restriction	MAST, 1,642 records	4 excluded from scoring	1,638 scored	The 4 fully-null records are excluded by the frozen manifest rather than deleted from the corpus
Taxonomy-version restriction	MAST human file, 19 records	15 excluded	4 usable	Only records labelled under the current 14-mode taxonomy are comparable
Pair extraction	QMSum, 232 meetings	0	1,810 pairs	Expansion, not filtering: one meeting yields many query-summary pairs

Table 19. Cleaning and validation rules, with the number of records each removed. No QMSum record is dropped; on the MAST path, exclusions are expressed in the frozen manifest rather than by deleting data, so an excluded record can still be inspected.

[OWNER: Naman] Add a before-and-after sample for the composite-key rule: one raw record with its ambiguous `trace_id`, and the same record after keying.

3.4 Data Transformation

Transformation is governed by versioned schemas rather than by convention, so that a candidate cannot quietly change the shape of what it emits. Two schemas are frozen for Model 1, and each transformation below exists for a stated reason.

`m1-input-v1` carries an `example_id`, a `session_id`, the participant's `query`, an ordered list of session events, and optionally a `memory_version` naming the approved memory items visible to that run. Events must have unique identifiers and strictly ascending unique sequence numbers. The reason is that a catch-up model that sees events in a different order from the session would produce a correct summary of a session that never happened, and the ordering constraint makes that unrepresentable rather than merely unlikely.

`m1-output-v1` requires the answer to be structured rather than free text. It has five categories — completed, in progress, blocked, decisions and pending actions — and every claim in every category must cite at least

one source event identifier. A validation step rejects any output whose claims cite an event the input did not contain. The reason is that an evidence link is only worth having if it is enforceable: without the check, a model could emit plausible identifiers and the citation would be decorative. A consequence worth stating is that approved memory can never become the evidence for a session fact. Memory may change how the model behaves, but it cannot be the source of a claim.

For the text-similarity metrics, the structured output is projected to canonical text by concatenating claims in a fixed category order with whitespace normalised. The reason is determinism: ROUGE and BERTScore are sensitive to ordering and spacing, so without a fixed projection the same structured answer could score differently on two runs.

A transformed record, drawn from `data/splits/m1_v1/test.jsonl`:

Field	Value (truncated)
`id`	`qmsum_test_13_s6`
`query`	Summarize the views to what extent they believe that universities understood the value of the Welsh bac.
`context_spans`	List of 12 transcript turns. First turn: "Janet Finch-Saunders AM: Okay. A question, then, just to ColegauCymru: to what extent do you believe that universities in the main understand the value of the Welsh bac?"
`reference_summary`	"They thought most universities did not understand, based on some experience they had with some of their learners that went to Russell Group universities. But this depended on the admission tutors and the areas that they were going…" (62 words)

Table 20. A transformed Model 1 record. The same four fields appear in every prepared split.

[OWNER: Ayush] Add the corresponding `m1-input-v1` and `m1-output-v1` pair once a session-shaped example exists. The record above is the QMSum supervision shape, not the session shape the product will serve.

3.5 Data Preparation

The Model 1 splits are fixed, version-controlled and checksummed. `data/splits/m1_v1/` contains the three split files and a `MANIFEST.json` recording the task, the contract name, the generation seed (13), and for each split a row count, a byte size and a SHA-256. Re-running the generator with the same seed reproduces the same files, and a changed file is detectable from the manifest alone.

One point of honesty about how these splits were formed. The generator does not re-partition QMSum. It processes each upstream split file separately and shuffles within it, so the partition is QMSum's own meeting-level partition, inherited rather than constructed. Nothing is stratified on any attribute. The generator exposes a grouping option for document or session splitting, but the Model 1 path does not read it, so selecting session grouping today would have no effect. This is a limitation to fix before the session-level splits are built, not a defect in the current QMSum splits, which inherit a partition that is already meeting-disjoint upstream.

Split	Rows	% of total	Stratified on	Leakage check and result
Train	1,257	69.4%	Not stratified; inherits QMSum's meeting-level partition (162 meetings)	Accounting reconciliation against the published corpus totals. Meetings 162 + 35 + 35 = 232, exactly the corpus total, and pairs 1,257 + 272 + 281 = 1,810, exactly the pair total. No meeting
Validation	272	15.0%	Not stratified; 35 meetings	

Split	Rows	% of total	Stratified on	Leakage check and result
Test	281	15.5%	Not stratified; 35 meetings	and no pair is therefore counted in two splits. **Result: passed, no leakage found.**

Table 21. The frozen `m1_v1` splits, their proportions, and the leakage check and its result.

The reconciliation above is the real check, and it is worth saying why the more obvious test would not have been one. The identifiers in the prepared files embed the split name, taking the form `qmsum_<split>_<meeting>_s<summary>`, so identifiers can never collide across splits and an identifier-overlap test across the prepared files is guaranteed to pass whether or not leakage exists. Such a test would report success while measuring nothing. The prepared records also do not carry the source meeting identifier as a field, so a meeting-level overlap test cannot be run on the prepared data at all without returning to the raw corpus. The counts reconciling exactly against the published corpus totals is what actually rules out an overlapping meeting.

Figure 4 shows the resulting split proportions and the meeting counts behind them. The validation and test splits are close in size, at 272 and 281 pairs, but both are drawn from only 35 meetings, which is the number that governs how much independent evidence a held-out result actually rests on.

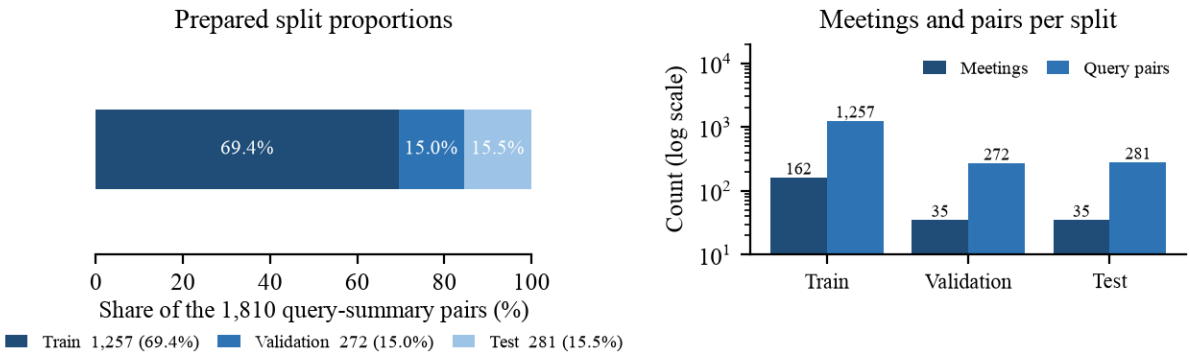


Figure 4. Frozen `m1_v1` split proportions, and the meetings and query pairs behind each split. Generated by `docs/reports/generate_t4a5_figures.py` from `data/splits/m1_v1/MANIFEST.json`.

[OWNER: Naman] Add one sample record from each of the three prepared splits. Table 20 gives the shape; three short examples are needed.

[OWNER: Naman, Ayush] The product-session split `m1_session_v1`, which the frozen contract names as the candidate-selection split, does not exist. It is the critical-path item for any reportable Model 1 comparison.

3.6 Data Statistics

Stage	Rows	Columns	Notes
Raw (MAST)	1,642 traces	6	`mas_name`, `llm_name`, `benchmark_name`, `trace_id`, `trace`, `mast_annotation`
Raw (QMSum)	232 meetings	3	Transcript, general query list, specific query list
Pre-processed (MAST)	1,642 retained, 1,638 scored	6 + composite key	No record deleted; 4 fully-null records excluded by the frozen manifest

Stage	Rows	Columns	Notes
Transformed (QMSum)	1,810 pairs	4	`id`, `query`, `context_spans`, `reference_summary`
Prepared (QMSum)	1,810 across three splits	4	1,257 / 272 / 281, each checksummed in `MANIFEST.json`

Table 22. Record and field counts at each stage of the pipeline.

Distribution of the Model 1 supervision data

Figure 6 reports the measured distributions of the prepared Model 1 data. Reference summaries are short and tightly clustered: the median is 66 words in train, 68 in validation and 59 in test, with a long right tail reaching 238 words. Queries are shorter still, with a median of 11 words. The evidence available per example varies far more than either, with a median of 29 to 33 context spans but a distribution that is heavily right-skewed, so a minority of examples require the model to read several times as much transcript as the typical case. Two consequences follow for Model 1. First, because summaries are short, a single omitted critical fact is a large proportion of the answer, which is the reason `critical_fact_recall` rather than a text-overlap score is ranked first in section 4.4. Second, because context length is skewed, a candidate's tail latency will be governed by the long-context minority rather than by the median, which is why latency is committed at p95 as well as p50.

The three split distributions sit almost on top of one another in all three panels. That is worth stating explicitly given section 3.5: the splits were inherited rather than stratified, so their distributional similarity is a property of the upstream partition that we checked, not one we engineered.

Statistic	Train	Validation	Test	All
Query-summary pairs	1,257	272	281	1,810
Source meetings	162	35	35	232
Summary length, median words	66	68	59	65
Summary length, mean words	70.6	70.6	64.7	69.6
Summary length, maximum words	238	188	188	238
Query length, median words	11	10	11	11
Context spans, median	29	30	33	30

Table 23. Measured statistics of the prepared Model 1 data, computed from `data/splits/m1_v1/` by `docs/reports/generate_t4a5_figures.py`.

Imbalance in the baseline corpus

The MAST corpus is markedly imbalanced along all three of its descriptive axes, which constrains what the reproduced baseline can claim. Of the seven frameworks, AG2 contributes 597 traces and MetaGPT 430, while OpenManus, AppWorld and HyperAgent contribute 30 each. Of the five underlying language models, GPT-4o accounts for 751 traces, with Qwen and CodeLlama at 200 each. Across the eight benchmarks, program-development tasks account for 790 of 1,642 traces between `ProgramDev` and `ProgramDev-v2`. Figure 5 shows all three axes.

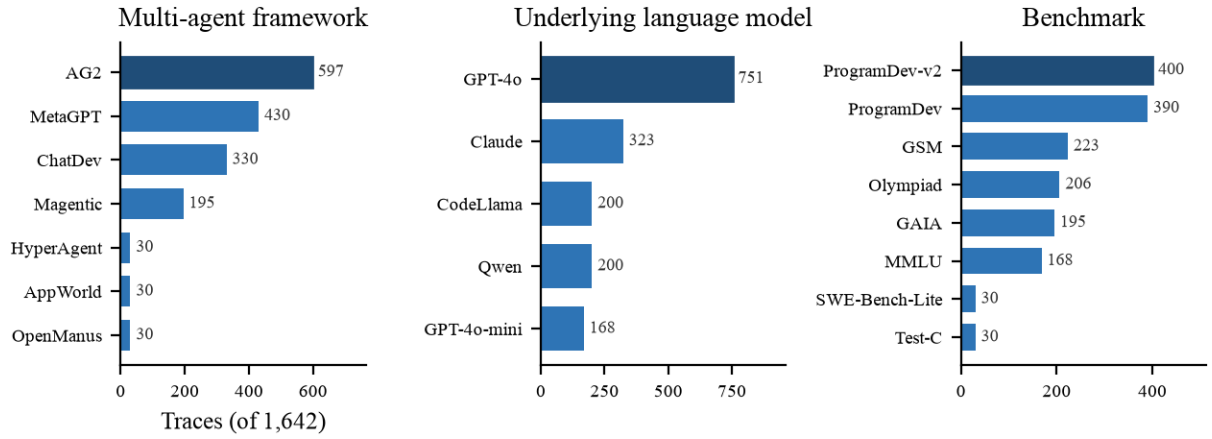


Figure 5. Composition of the 1,642-trace MAST corpus across framework, underlying language model and benchmark. Generated by `docs/reports/generate_t4a5_figures.py` from `data/mast/DATA_DICTIONARY.md`.

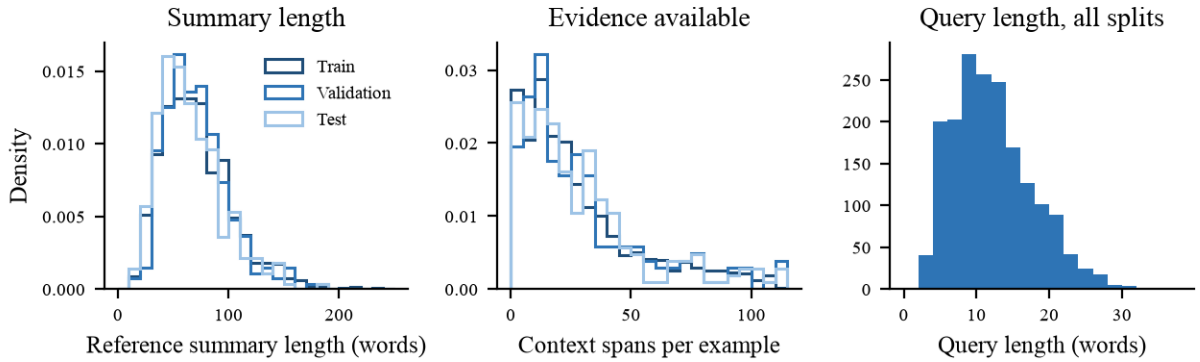


Figure 6. Measured distributions of the prepared Model 1 data: reference summary length, context spans available per example, and query length. Generated by `docs/reports/generate_t4a5_figures.py` from `data/splits/m1_v1/`.

Axis	Distinct values	Largest	Smallest
Framework (`mas_name`)	7	AG2, 597	OpenManus, AppWorld, HyperAgent: 30 each
Language model (`llm_name`)	5	GPT-4o, 751	GPT-4o-mini, 168
Benchmark (`benchmark_name`)	8	ProgramDev-v2, 400	Test-C, SWE-Bench-Lite: 30 each

Table 24. Class imbalance in the MAST corpus across its three descriptive axes.

Population and coverage gaps

Three gaps are worth stating plainly, because each would disadvantage an identifiable group if the system were deployed on this evidence alone.

First, QMSum is transcribed speech from academic, product and technical meetings conducted in English. A catch-up model supervised only on it is untested for speakers of other languages, for non-native English speakers whose transcripts carry more disfluency and self-correction, and for domains such as clinical or legal work where a missed blocker carries far greater cost than it does in a product meeting. The people

disadvantaged are precisely those whose speech the upstream transcription and annotation pipeline handled least well, and nothing in our pipeline corrects for that.

Second, no product-session data exists yet, so every Model 1 number available today is measured on meeting transcripts rather than on the multiplayer sessions the product actually serves. A meeting transcript has one speaker at a time and no agent actors; a session has several humans and an agent acting concurrently. Transfer between the two is assumed and has not been measured, so a reader should treat current Model 1 evidence as evidence about meeting summarization.

Third, the MAST imbalance above means the reproduced baseline speaks mainly to AG2 and MetaGPT traces driven by GPT-4o on program-development tasks. Its failure-detection behaviour on the three 30-trace frameworks is not meaningfully measurable at that sample size and should not be reported as if it were. Teams using those frameworks would be the ones misled by a headline accuracy figure.

4. Model Development

4.1 Model Proposals

Four models are built by this team, one in 298A and three in 298B, and one published baseline is reproduced for reference. The baseline is not one of the four; it is the external reference point against which the evaluation tooling is validated. Each of the four addresses a distinct failure of single-user agent tooling identified in section 1.

#	Model	Concept and architecture	Built in	Owner
1	M1, session-state catch-up summarizer	Produces a structured, evidence-linked catch-up for a participant joining a session late. Output is five claim categories, each claim citing the session events that support it. Selected from a four-candidate bake-off spanning a non-neural control, an extractive selector and two QLoRA-tuned generative models [10], [18].	298A	Ayush Gawai
2	M2, intent and addressee attribution	Determines which participant a given instruction is meant to serve, so that concurrent instructions from several humans can be attributed before they are acted on. Cross-encoder classification over the instruction and the candidate addressees [11], [15].	298B	Naman Vipul Chheda
3	M3, merge arbitration	Arbitrates between conflicting concurrent instructions over a four-way decision: accept A, accept B, merge, or escalate to a human. Last-writer-wins is retained as the control arm.	298B	Manav Patel
4	M4, task decomposition and routing	Plans and routes tool execution, producing a valid task DAG restricted to the five fixed tools the requesting participant is eligible to invoke. DAG validity gates eligibility before any quality metric is read.	298B	Shriram Dundigalla
—	Reproduced baseline: MAST LLM-as-a-Judge	The published MAST failure taxonomy and its validated LLM judge [3], reproduced on the pinned 1,642-trace release in the paper's o1 few-shot configuration. Target values are accuracy 0.94, F1 0.80 and Cohen's kappa 0.77, from Table 2 of [3].	298A	Pramod Satya Dindukurthi, with Shriram Dundigalla (runner and provenance) and Manav Patel (prompt, parser, viewer)

Table 25. The four models built by this team and the single published baseline reproduced for reference. The baseline is not one of the four.

The four candidates evaluated for Model 1 are a separate matter from the four models above, and the two should not be confused. Model 1 is one of the four models; Table 26 lists the four competing architectures from which that single model is selected. No architecture is pre-chosen, and the winner is decided by the frozen rule in section 4.4.

Candidate	Architecture	Why it is in the matrix
`m1-a`	BM25 event ranking with a fixed structured template [22]	Transparent, CPU-only, non-neural floor. Establishes whether a learned model is needed at all, and is structurally incapable of inventing a claim because it can only quote.
`m1-b`	ModernBERT-base extractive event selector with a fixed template [15]	Tests whether learned extraction alone closes the gap without generation, at encoder cost rather than generator cost.
`m1-c`	Phi-4-mini-instruct with QLoRA [18], [23]	Compact generative model from a different pretraining family; tests evidence-linked abstractive summarization at moderate GPU cost.
`m1-d`	Qwen3.5-4B with QLoRA [16], [18]	Larger generative candidate; tests whether added capacity justifies its latency and VRAM.

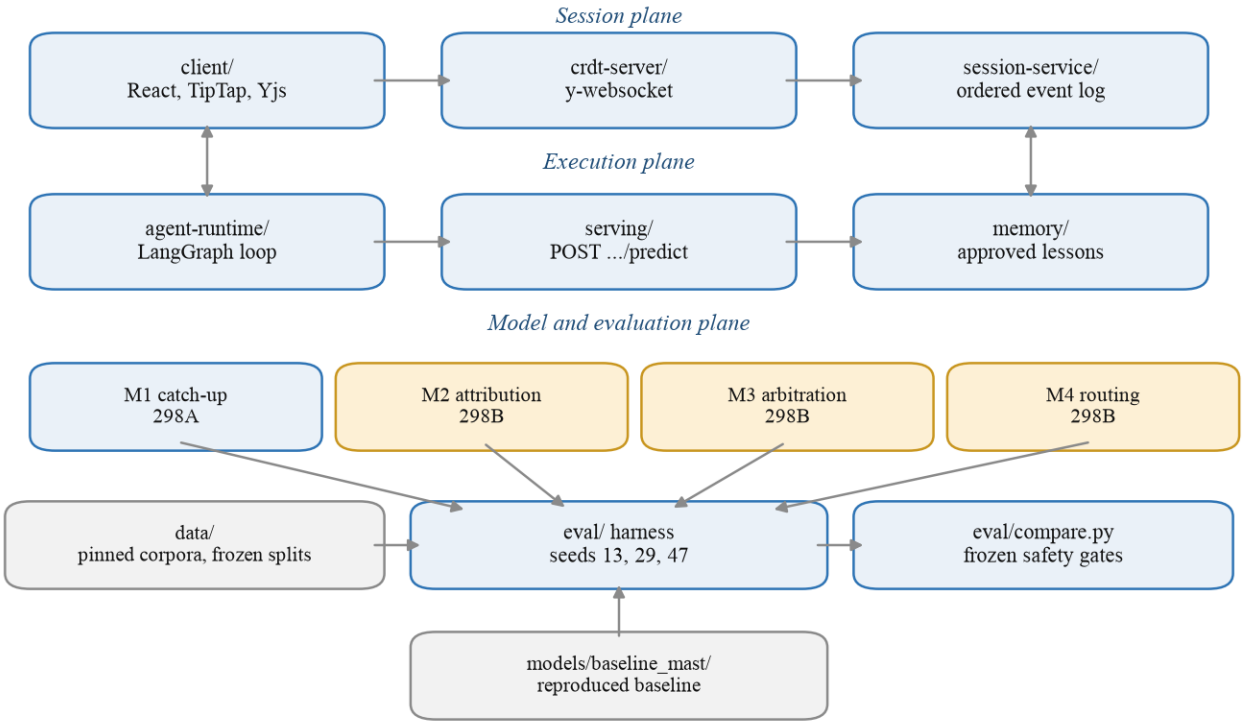
Table 26. The four-candidate matrix for Model 1, frozen in ADR-005.

4.2 Model Supports

The models are developed and executed on a platform built by this team rather than in isolated notebooks, because the evidence-linking requirement means a model is only meaningful alongside the session event log whose events it cites. Figure 7 shows the components, the machine-learning data flows between them, and where the frozen contracts sit.

Component	Role	Technology
`session-service/`	Ordered session event log and HTTP API. The event contract is frozen in ADR-001 and its persistence in ADR-002.	FastAPI, Pydantic, SQLite, PostgreSQL
`serving/`	Uniform predict interface for every model, `POST /v1/models/{model_id}/predict`, so the harness calls all candidates identically and no candidate can report through its own code path.	FastAPI
`eval/`	The only place metrics are computed. Shared harness, deterministic seeding, per-seed provenance and prediction checksums, and the fail-closed candidate comparison.	Python, NumPy
`agent-runtime/`	Plan, act, observe agent loop with a hard step cap, failed-tool handling and cancellation.	LangGraph [13]
`memory/`	Approved-memory store: proposed, approved and rejected lesson states, approved-only retrieval, append-only audit log, and rollback by memory version.	Python, Pydantic, POSIX file locking
`client/`, `crdt-server/`	Multiplayer editing surface with convergent document state and participant presence.	React, TipTap, Yjs [5], [6]
Training environment	QLoRA fine-tuning with checkpointing and resume, verified end to end on the university GPU lab before the bake-off opens.	PyTorch, Transformers [24], PEFT [25], bitsandbytes
`infra/`	Reproducible local bring-up of the services above.	Docker Compose

Table 27. Platform components supporting model development and execution.



Amber: 298B. The baseline is scored by the same harness as the four models and is never one of them.

Figure 7. Supporting platform architecture and machine-learning data flow. The baseline is scored by the same harness as the four models and is never one of them. Generated by `docs/reports/generate_t4a5_figures.py`.

4.3 Model Comparison and Justification

The four models differ in what they take as input, in the class of method they need, and in what limits them. They are compared below on those terms rather than on expected accuracy, since none has a result yet and a comparison of anticipated numbers would be fiction.

Model	Targeted problem	Features	Approach class	Strengths	Limitations	Justification
M1	A participant joining late cannot cheaply learn what has happened	Ordered session events with actor, type and target; the participant's query; approved memory version	Deep learning (generative, adapter-tuned), with a non-neural retrieval control in the matrix	Every claim cites a source event, so the output is checkable rather than merely fluent; public supervision exists, so it can be developed before product sessions do	Supervised on meeting transcripts, not sessions, so transfer is assumed; generation cost grows with session length; a fluent model can still invent a claim	Catch-up is the failure every participant hits, its output is read directly by a human, and it is the only one of the four with public supervision available in 298A
M2	It is unclear which participant a given instruction is meant to serve	Instruction text, speaker, recent addressee history, participant roles and presence	Machine learning: cross-encoder classification, with a rule-based	Short inputs and cheap inference; labels are observable from the session itself at annotation time	The ceiling is annotator agreement, which Workbook 1 already identifies as the expected	Attribution is a precondition for arbitration: M3 cannot decide between two instructions until

Model	Targeted problem	Features	Approach class	Strengths	Limitations	Justification
			mention control	rather than requiring judgement about outcomes	limiting factor; depends entirely on a pilot corpus that does not exist yet	it knows whom each is for
M3	Two actors redirect the same agent at once and nothing arbitrates	The two competing instructions, their attributed addressees, the document state diff, and actor roles	Machine learning: four-way classification with abstention, against a last-writer-wins control	The control arm is the status quo, so any improvement is measured against what the product would otherwise do; escalation gives the model a safe action instead of a forced choice	Harmful merges are rare, so the class that matters most is the least observed; a model that always escalates scores well on harm and badly on usefulness	This is the decision today's products make silently and sometimes wrongly, and making it explicit is the core claim of the project
M4	Tool execution must be planned and bounded by permission	Task description, tool catalogue, participant permissions, current workload	Deep learning planner with a learned router, gated on DAG validity; rules-only control	Validity is checkable mechanically before any quality metric is read, so an invalid plan can never win on quality	Five fixed tools make the routing space small, so a rules-only control may be hard to beat; that is a genuine risk to the contribution claim rather than a formality	Routing is what turns the other three models into actions, and permission-bounded execution is what makes interruption and rollback specifiable

Table 28. Comparison and justification across the four models. The reproduced baseline is excluded because it is a reference point, not a candidate.

When the preferred model would lose

For Model 1 the preferred arm is a QLoRA-tuned generator, and four conditions would make one of the other three the better choice. Each is a real possibility rather than a hedge.

The first is tail latency. A catch-up is read on arrival, so a participant who is waiting will abandon it. If `latency_p95_ms` for a generative candidate exceeds the point at which a participant stops waiting, the BM25 control wins on the only dimension that matters at that moment, regardless of its lower recall. Section 3.6 shows why this is a live risk: the context-span distribution is heavily right-skewed, so the long-context minority, not the median example, sets the tail.

The second is deployment cost. If the deployment environment cannot host a four-billion parameter model, `m1-a` or `m1-b` wins by default. This is why `peak_vram_mb` is committed as a tie-break metric in section 4.4 rather than being reported as an afterthought.

The third is the faithfulness gate, and it is the most interesting of the four. The selection rule disqualifies any candidate whose `unsupported_claim_rate` exceeds 0.02. The non-neural control cannot fail that gate, because it can only quote events that exist; a generative candidate can fail it at any quality level. A generator that is better on every quality metric and fails the gate does not win, and that outcome is intended rather than accidental.

The fourth is session length. On a short session with few events, a template filled by extractive selection may match a generator's critical-fact recall at a fraction of the cost, because there is little to compress. If

that is what the bake-off finds, the correct conclusion is that Model 1 should be a learned model only above a session-length threshold, and the report will say so rather than reporting a single winner across all conditions.

4.4 Model Evaluation Methods

The metrics below are committed in advance of any result, in the frozen experiment contract `eval/configs/m1-contract.yaml` and in ADR-005, dated 22 October 2026. They are not revised after results are seen. Three seeds, 13, 29 and 47, are fixed by the same contract, and a candidate must complete all three to be eligible for selection.

Metric	Why this metric	Benchmark or split	Committed on
`schema_valid_rate`	An output that does not parse cannot be used by the product, so validity is a precondition rather than a quality score.	`session_test`	2026-10-22
`critical_fact_recall`	Primary quality metric. A catch-up that omits a blocker has failed at its purpose, so coverage of gold critical facts is ranked first.	`session_test`	2026-10-22
`unsupported_claim_rate`	Faithfulness. A confident claim with no supporting event is the most damaging failure mode for a late joiner, who has no way to detect it.	`session_test`	2026-10-22
`attribution_accuracy`	Evidence links must point at the right events, not merely at some event, or the citation is decorative.	`session_test`	2026-10-22
`status_consistency`	Placing a blocked item under completed misleads precisely the reader the model serves.	`session_test`	2026-10-22
`rouge_l`	Comparability with the summarization literature [19]. Convention `m1_rouge_macro_v1`, locked in ADR-004.	`qmsum_test`, `session_test`	2026-10-22
`bertscore_f1`	Semantic similarity where wording differs legitimately [20]. Pinned to `bert-score==0.3.13`, `roberta-large` [21] at a fixed revision, 17 layers, no IDF, no baseline rescaling.	`qmsum_test`, `session_test`	2026-10-22
`human_edit_rate`	Reviewer effort to make an output acceptable, as a check that the automatic metrics track usefulness.	`session_test`	2026-10-22
`latency_p50_ms`, `latency_p95_ms`	A catch-up is read on arrival; tail latency decides whether it is read at all.	Measured after warmup on the evaluation split	2026-10-22
`peak_vram_mb`	Deployment cost, and the tie-break between candidates of comparable quality.	Measured during inference	2026-10-22

Table 29. Evaluation metrics for Model 1, committed before any results existed.

Safety gates and the selection rule

Eligibility is decided before ranking, and a candidate that fails any gate cannot win regardless of its quality scores. A candidate must complete all three seeds, reach at least 0.98 schema validity, keep unsupported

claims at or below 0.02 and no worse than the non-neural control, and reach at least 0.95 on both attribution accuracy and status consistency. Eligible candidates are then ranked by critical fact recall, then BERTScore-F1, then ROUGE-L, with latency and peak VRAM as tie-breaks. The machine-ranked leader is provisional: the contract requires blinded human review of the top two before a winner is declared, and if no candidate passes every gate, no winner is selected. Figure 8 shows the rule in order.

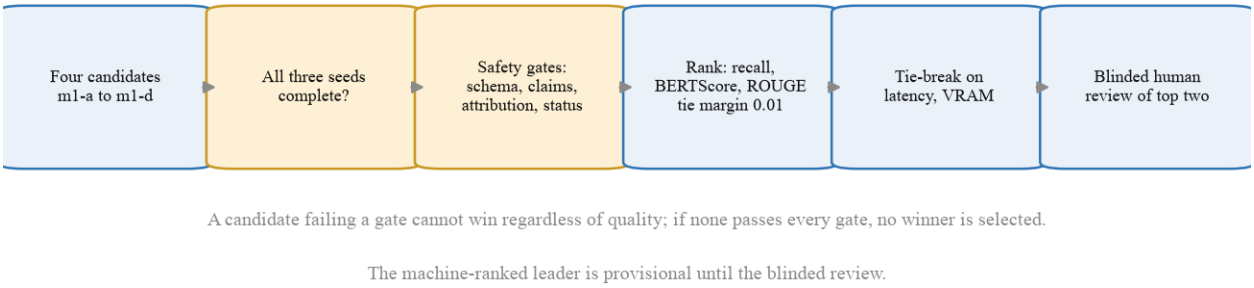


Figure 8. The frozen Model 1 selection rule: completeness and safety gates decide eligibility before any quality ranking is read. Generated by `docs/reports/generate_t4a5_figures.py`.

Models 2 through 4 are evaluated in 298B against the thresholds already committed in Workbook 1, Table 12: macro-F1 at or above 0.75 for M2, macro-F1 at or above 0.70 for M3 under harmful-merge and escalation guardrails, and for M4 a DAG-validity gate followed by routing accuracy, completion, latency and cost against the rules-only control. Those commitments are unchanged and are not restated as new here.

Evaluation of the reproduced baseline is separate and uses the MAST taxonomy's own fourteen failure codes, scored against the manifest's reference sources, reported per reference source and never pooled. Its target values are the published accuracy 0.94, F1 0.80 and Cohen's kappa 0.77 from Table 2 of [3], with sample size and a 95% confidence interval reported beside every value.

4.5 First Model Implementation

This is DATA 298A work. Per the Workbook 2 template it is written up in the final report rather than in this workbook, because the first model is still running at the time of submission.

4.6 Baseline Reproduction

This is DATA 298A work. Per the Workbook 2 template it is written up in the final report rather than in this workbook, because the reproduction is still running at the time of submission.

References

References [1] through [17] are listed in Workbook 1 and are not repeated. The entries below are the sources newly introduced in sections 3 and 4.

[18] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36*, 2023.

[19] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. Workshop on Text Summarization Branches Out (ACL)*, 2004, pp. 74-81.

[20] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2020.

[21] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[22] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333-389, 2009.

[23] Microsoft, "Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs," arXiv:2503.01743, 2025.

[24] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020, pp. 38-45.

[25] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, "PEFT: State-of-the-art parameter-efficient fine-tuning methods," GitHub repository, 2022. [Online]. Available: https://github.com/huggingface/peft. [Accessed: Nov. 11, 2026].

Self-assessment and gap register

Report section	Our rating	If below good: what is missing	Owner	Lands by
3.1 Data Process	good	Complete, with Figure 3.	Naman Chheda	Landed
3.2 Data Collection	fair	Sources, pinning and quantities are complete; raw sample records are still to be pasted.	Naman Chheda	[date]
3.3 Data Pre-processing	good	Rules and counts are reported; one before-and-after sample would complete it.	Naman Chheda	[date]
3.4 Data Transformation	good	The QMSum transformed sample is real; the session-shaped `m1-input-v1`/`m1-output-v1` pair cannot exist until sessions do.	Ayush Gawai	[date]
3.5 Data Preparation	good	Splits frozen and checksummed, leakage check reported with its result. Per-split sample records outstanding.	Naman Chheda	[date]
3.6 Data Statistics	good	Complete, with measured statistics and two figures from real data.	Naman Chheda	Landed
4.1 Model Proposals	good	Complete.	Ayush Gawai	Landed
4.2 Model Supports	good	Complete, with Figure 7.	Ayush Gawai	Landed
4.3 Model Comparison	good	Complete, including the conditions under which the preferred model loses.	Ayush Gawai	Landed
4.4 Evaluation Methods	good	Complete and committed before results.	Pramod Satya Dindukurthi	Landed
4.5 First Model Implementation	fair	The runtime and event schema are implemented, tested and merged. Approved-memory retrieval (DAT-56) is written with passing tests but is still in review and not merged, so the component table in 4.2 describes it as designed rather than as landed. Either way the section reports an implementation rather than a trained Model 1: no candidate has been fine-tuned and no validation number exists yet.	Shriram Dundigalla	[date]

Report section	Our rating	If below good: what is missing	Owner	Lands by
4.6 Baseline Reproduction	poor	Runner, provenance, prompt, parser and viewer are complete and the record shape is frozen in ADR-003, but the judge has not been run, so the section states a reproduction procedure with no reproduction result against the published 0.94 accuracy.	Pramod Satya Dindukurthi	[date]
Product-session dataset	poor	Not a report section but the binding constraint on both. `m1_session_v1` does not exist, so no Model 1 comparison can be reported on the split the contract names.	[Naman / Ayush]	[date]

Carry-forward from Workbook 1

Item	Why it did not land, or what changed	Owner	New target
Model 1 candidate matrix	Changed. Workbook 1 Tables 1 and 8 named an earlier mapping; the approved matrix is now BM25, ModernBERT-base, Phi-4-mini-instruct with QLoRA, and Qwen3.5-4B with QLoRA, frozen in ADR-005. Table 26 supersedes Workbook 1 on this point.	Ayush Gawai	Landed 2026-10-22
QMSum pair count	Corrected. Workbook 1 stated 1,808 query-summary pairs in DR-1 and in the literature table; the measured count on the pinned download is 1,810. Section 3.2 uses the measured figure.	Naman Chheda	Corrected here
MAST reproduction result	Did not land. The runner, provenance, prompt, parser and viewer are complete and the record shape is frozen in ADR-003, but the judge has not been run, so no reproduction number exists.	Pramod Satya Dindukurthi	[date]
Product-session dataset	Did not land. Workbook 1 section 2.1 recorded the pilot corpus as planned with no sessions recorded; that is still true. The frozen contract names `session_test` as the candidate-selection split and `data/splits/m1_session_v1/` does not exist.	[Naman / Ayush]	[date]
Session-level split checks	Partly landed, with a caveat. Workbook 1 DR-4 promised an automated session-level leakage check. `make_splits.py` accepts a grouping option but the Model 1 path does not read it, so the guarantee currently rests on QMSum's inherited meeting-level partition and the reconciliation in Table 21, not on a grouping implementation.	Naman Chheda	[date]
M1 held-out ROUGE-L target of 28.2	Unchanged as a reference floor. Workbook 1 noted this is a QMSum reference figure and not a result this team has posted; it remains unposted.	Ayush Gawai	[date]
[ ]	[OWNER: all] Any other section 1 or 2 text revised since Workbook 1 that is not listed above	[ ]	[ ]